```
""" Production-Grade File Engineering in Python """

import os
import shutil
from pathlib import Path


# End-to-End GenAI & Data Science Series
# Focus: I/O, Directory Management, Archives, & Persistence
```

The Data Science File Ecosystem

File Type	Python Module	Typical Use in Data Science / GenAI
Text (.txt, .log)	`open()`	Logs, plain prompts
CSV / Parquet	`csv`, `pandas`	Tabular datasets, feature matrices
JSON / XML	`json`, `xml.etree`	API payloads, LLM prompts
ZIP / TAR	`zipfile`, `shutil`	Model checkpoints, dataset distribution
Pickle (.pkl)	`pickle`	Object serialisation (models, dicts)

Anatomy of a Safe Operation

Fragile

```
f = open(...)
content = f.read()
f.close()
```

Resource Leak Risk: If an exception occurs here, the file descriptor is never released.

Robust

```
with open(...) as f:
    content = f.read()
```

✅

Anatomy of a Context Manager

with block entered
(Lock Acquired)

Operations executed
(Data read/written)

Block exited
(Lock Automatically
Released via __exit__)

File Open Modes Diagnostic

Mode	Creates if Absent?	Truncates Existing?	Allows Read?	Allows Write?
r	✗	✗	✓ Raises FileNotFoundError if file absent.	✗
w	✓	✓ Truncates existing content.	✗	✓
a	✓	✗	✗	Append
r+	✗	✗	✓	✓
x	✓	N/A	✗	✓ Raises FileExistsError if exists

Binary Variants: Append ‘b’ (e.g., ‘rb’, ‘wb’) to any mode for images, pickles, and ZIP files.

Flat Discovery: Filtering Directories

String Methods (Built-in)

```
entries = os.listdir('datasets')
[f for f in entries if
  f.endswith(('.csv', '.json'))]
```

Efficient Filtering
Tuple acceptance eliminates the need for complex any() conditions.

Low-Level Patterns (fnmatch)

```
fnmatch.filter(entries, 'report_2024_*.csv')
```

*	: Any sequence
?	: Single character
[seq]	: Any char in sequence
[!seq]	: Any char NOT in sequence

Deep Discovery: Recursive Search

Legacy Approach

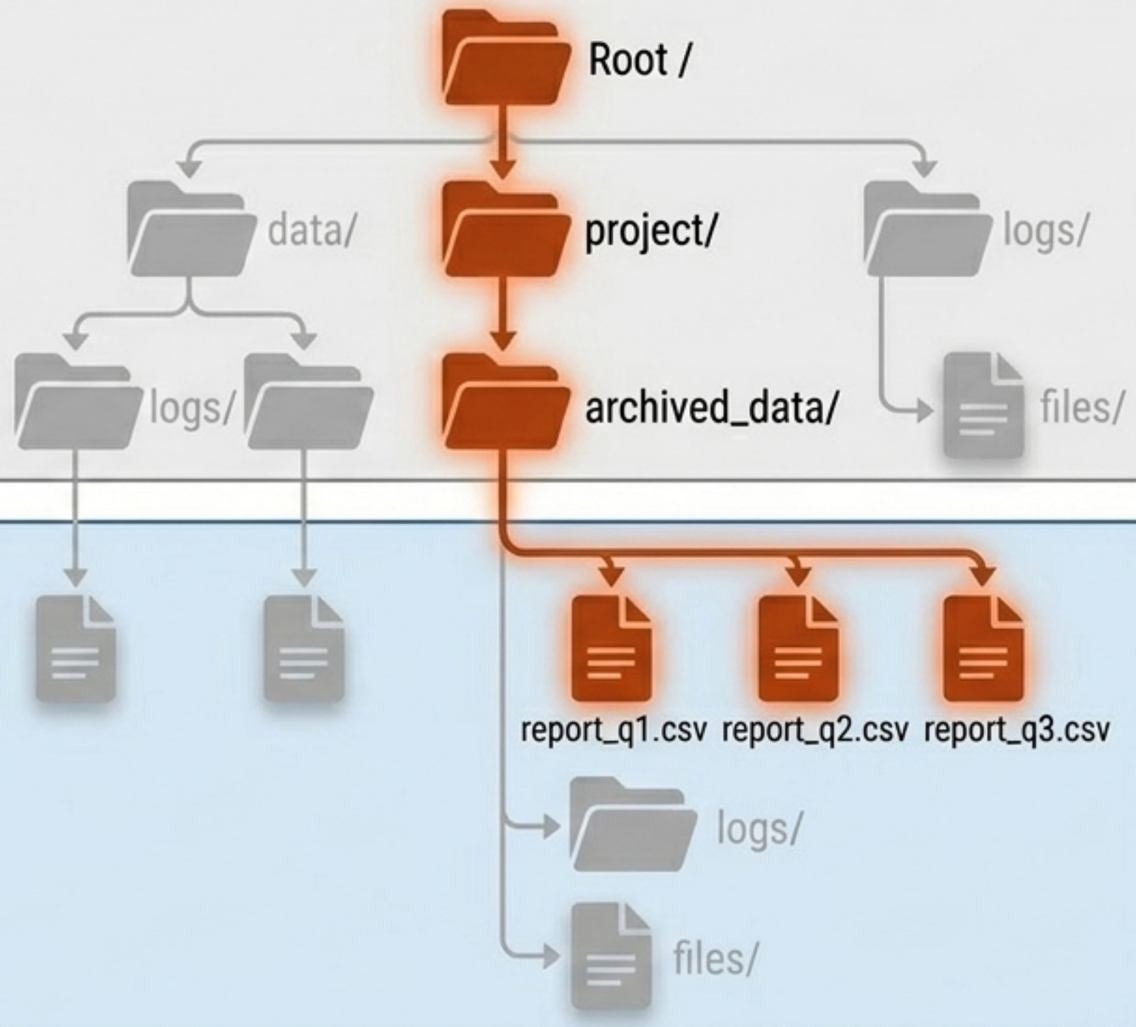
```
glob.glob('**/*.csv', recursive=True)
```

Requires manual recursion flags and returns fragile string paths.

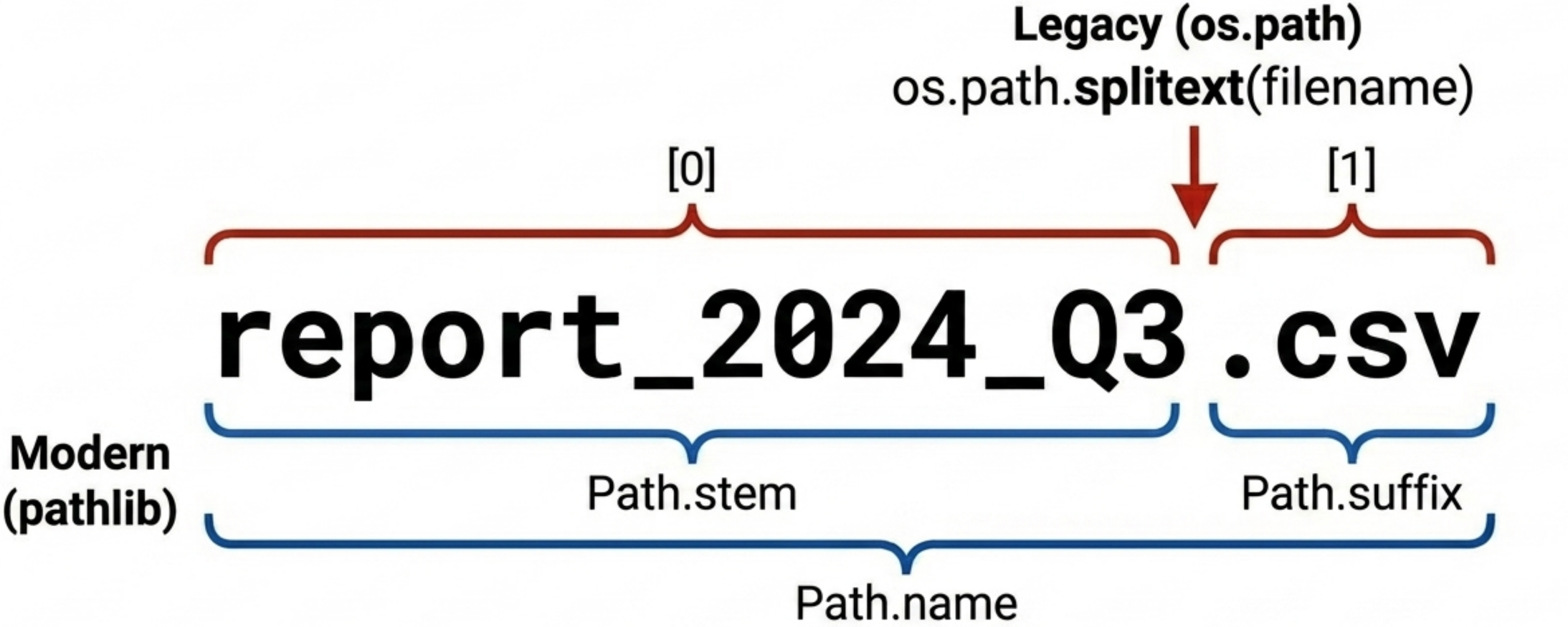
Modern Mandate (pathlib)

```
list(Path('.').rglob('*.csv'))
```

Returns chainable Path objects. Integrates instantly with the rest of the API.

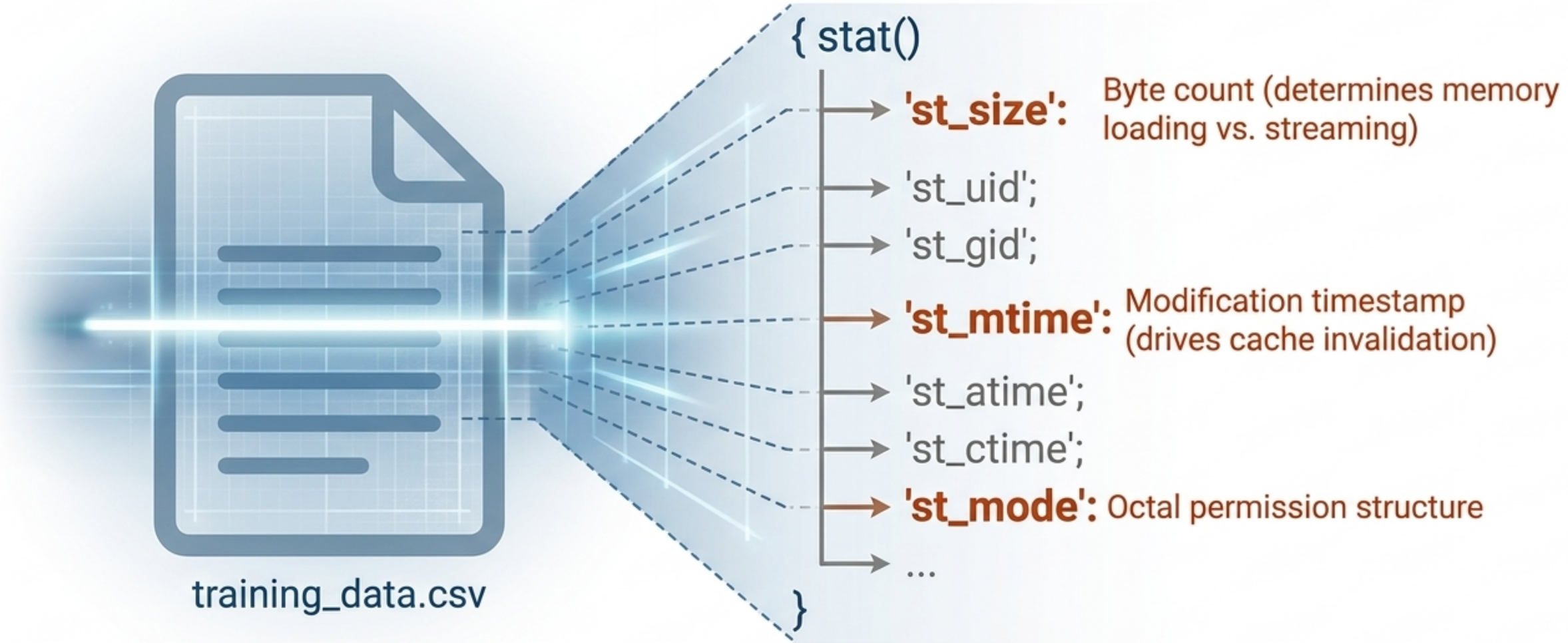


Deconstructing Filenames



Stop slicing strings manually. Treat paths as semantic objects.

Querying Metadata: Anatomy of a File




```
stat = Path('training_data.csv').stat()
```

The Rosetta Stone: os.path to pathlib

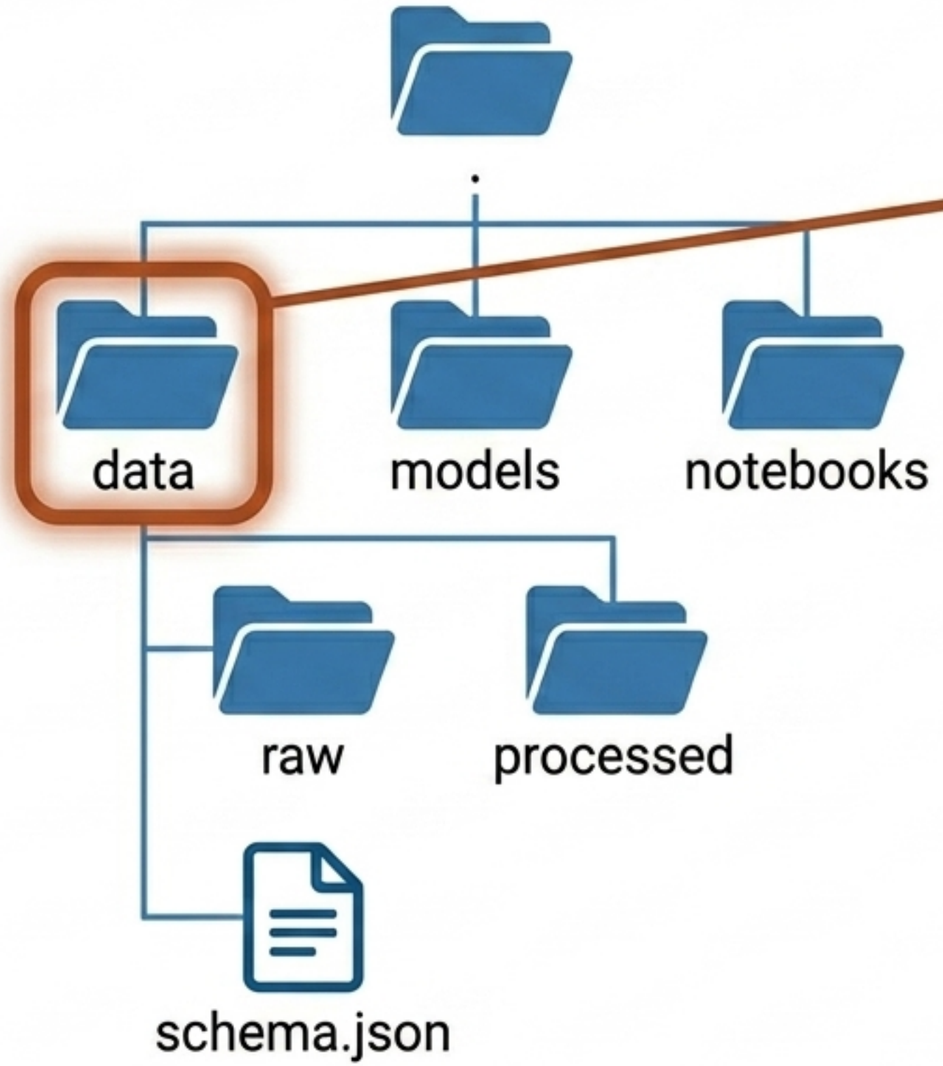
Legacy (os.path)	Modern (pathlib)
`os.path.exists(p)`	→ `Path(p).exists()`
`os.path.isfile(p)`	→ `Path(p).is_file()`
`os.path.getsize(p)`	→ `Path(p).stat().st_size`
`os.path.abspath(p)`	→ `Path(p).resolve()`
`os.path.join(a, b)`	→ `Path(a) / b`

Design Principle:

Prefer **pathlib** for all path manipulation in new code.

The 'os' module remains necessary only for **process-level operations**.

Demystifying os.walk()



The Yielded 3-Tuple

- **root**: `'./data'`
- **dirs**: `['raw', 'processed']`
- **files**: `['schema.json']`

Pro Tip

```
dirs[:] = [d for d in dirs if d not in {'.git'}]
```

Modifying `dirs[:]` in-place prunes the search tree. This excludes caches and virtual environments for speed and correctness.

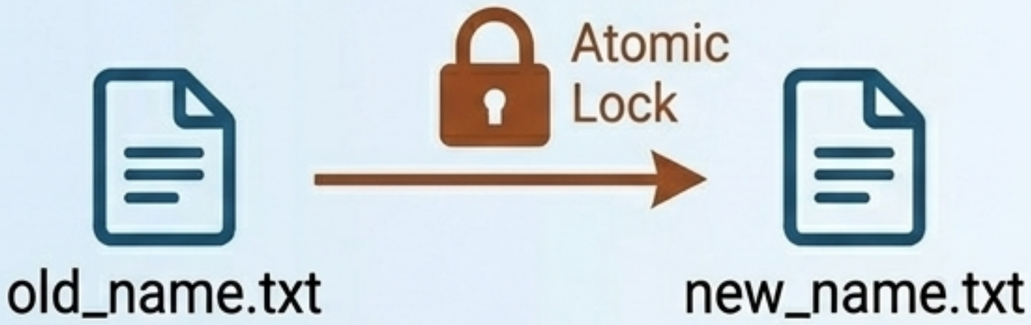
High-Level File Operations (shutil)

Function	Copies Content	Preserves Metadata (Timestamps)	Preserves Permissions
`shutil.copy(src, dst)`	✅ Yes	❌ No	✅ Yes (Fastest, most common)
`shutil.copy2(src, dst)`	✅ Yes	✅ Yes	✅ Yes (Preferred for precise backups)
`shutil.copytree(src,dst)`	Recursive copy of an entire directory tree.		

Raw '**os**' system calls miss cross-device move edge cases.
'**shutil**' handles these gracefully.

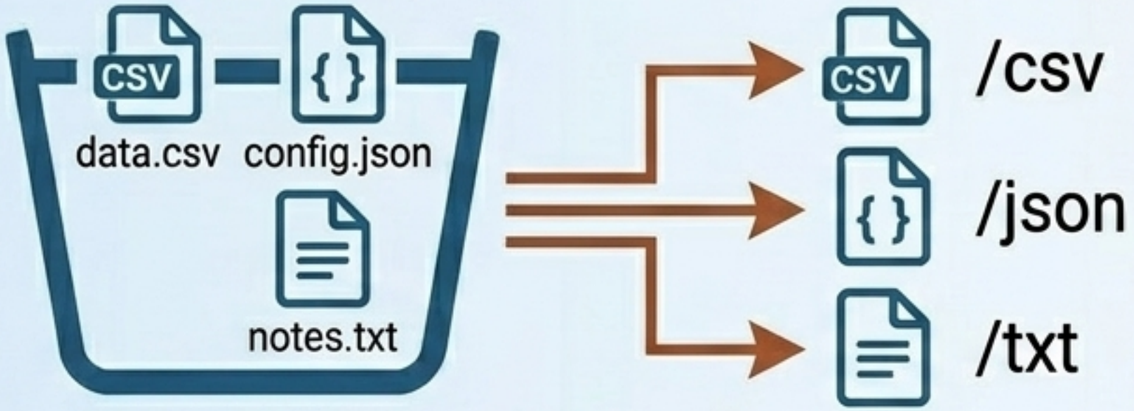
Safe Renames and Moves

Atomic Operations (`Path.rename`)



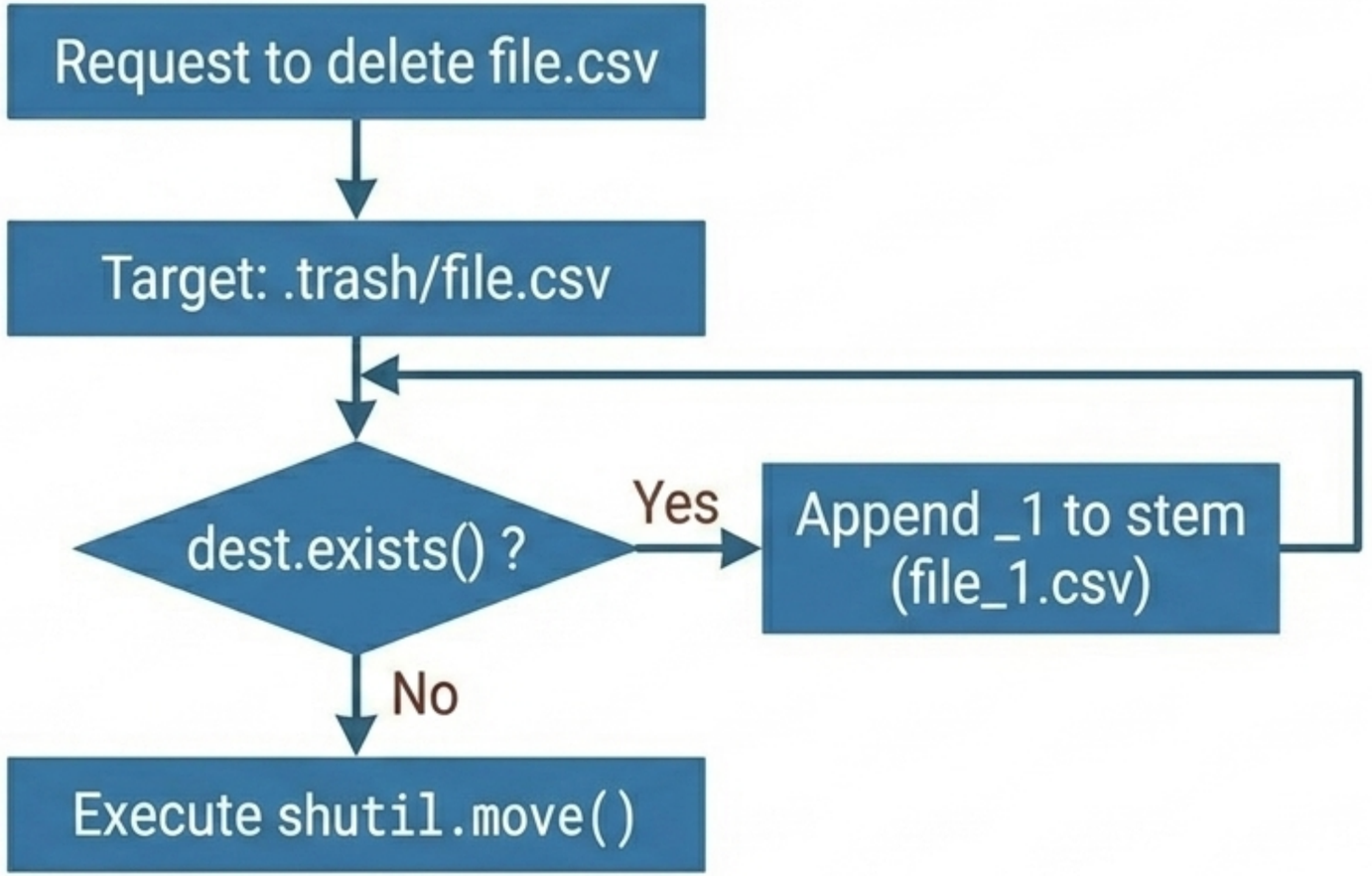
Instantaneous, all-or-nothing operation when on the same filesystem. Safe for concurrent production environments.

Production Pattern: Type Sorting (`shutil.move`)




```
dest.mkdir(parents=True)
shutil.move(src, dest / src.name)
```

Defensive Engineering: Safe Deletion



SAFETY RULE: Never call `shutil.rmtree()` without validating the path with `is_relative_to(expected_root)`. A bug resolving to '/' will wipe the filesystem.

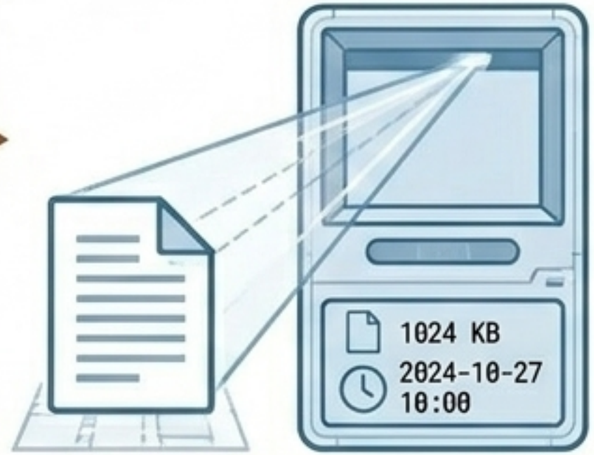
Applied Pattern: Production Data Inventory

1. Discovery

```
Path(dir).rglob('*.csv')
```

2. Metadata Extraction

```
path.stat()
```



3. Synthesis

```
{
  'name':
    path.name,
  'size_bytes':
    stat.st_size
}
```

This pattern forms the foundation of incremental data processing pipelines, allowing systems to audit directories, record sizes, and identify stale files dynamically.

The Modern File Engineering Mandate

1. **Always use Context Managers**: Never open a file without a `'with'` block. Resource leaks are unacceptable.
2. **Pathlib is Default**: Treat paths as objects, not strings. `'os.path'` is legacy.
3. **Inspect Before Ingest**: Use `'stat()'` to determine file size before naively loading massive datasets into memory.
4. **Soft Deletes Over Permanent Wipes**: Build trash archives with collision-handling loops instead of relying on `'os.remove()'`.